UCLouvain

LINFO1361 - Intelligence artificielle

Projet 2 - Recherche adversariale (Oxono)

Groupe n°114
Röben Friedrich Nihat 99942500
Sogliuzzo Loris 26482510

ÉCOLE POLYTECHNIQUE DE LOUVAIN

Table des matières

1 Introduction **4**

2 Implémentation **4**
2.1 Minimax 4
2.2 Monte-Carlo Tree Search (MCTS) 4
2.3 AlphaZero 4
2.4 Athénan 5

3 Résultats et discussions **6**
3.1 Tournoi suisse 6

4 Conclusion **7**

Table des figures

Liste des tableaux

1 Architecture du réseau de neurones *R2-network* implémenté. . 5
2 Résultats détaillés du tournoi suisse (4 rondes, 20 parties par match). 6

1 Introduction

2 Implémentation

2.1 Minimax

2.2 Monte-Carlo Tree Search (MCTS)

2.3 AlphaZero

En plus des agents guidés par des heuristiques faites à la main, nous avons voulu implémenter des agents basés sur des réseaux neuronaux. En premier lieu, nous avons choisi d'adapter l'implémentation d'AlphaZero (Silver et al., 2016, 2017) à Oxono.

Pour ce faire, nous avons implémenté le même réseau de neurones que celui utilisé par AlphaZero, à savoir un réseau de neurones convolutifs avec des blocs résiduels possédant deux têtes de sortie : une tête de politique pour prédire la probabilité de chaque action et une tête de valeur pour prédire l'évaluation de l'état.

De plus, il a fallu adapter la représentation de l'état du jeu pour qu'elle soit compatible avec le réseau de neurones. Nous avons utilisé un tenseur comportant 7 canaux, chacun de dimension 6×6, correspondant à la taille du plateau de jeu, où les canaux 1 et 2 encodent respectivement les positions des pions O de couleurs rose et noire, 3 et 4 sont dédiés aux X, 5 et 6 encodent la position des totems O et X, et le dernier canal encode le tour de jeu.

Ensuite, pour la sélection d'action, nous avons couplé le réseau de neurones à l'algorithme MCTS. La sortie de la tête de politique sert à guider la recherche, tandis que la sortie de la tête de valeur sert à évaluer les nœuds feuilles de l'arbre. Durant les simulations, la sélection des nœuds s'effectue selon la formule PUCT (Predictor + Upper Confidence Bound applied to Trees) (équation 1) :

$$PUCT(s,a) = -Q(s,a) + c \cdot P(s,a)\frac{\sqrt{N(s)}}{1+N(s,a)} \tag{1}$$

Où $Q(s,a)$ est la valeur moyenne de l'action a évaluée depuis le point de vue du joueur suivant, $P(s,a)$ est la probabilité de l'action a donnée par la tête de politique, $N(s)$ est le nombre total de visites de l'état s, et $N(s,a)$ le nombre de visites de l'action a. L'hyperparamètre c contrôle l'exploration, il est fixé à 1.5 lors de la phase de jeu, mais augmenté à 2.5 durant l'entraînement pour garantir une meilleure diversité d'exploration.

Afin d'entraîner ce réseau de neurones sans aucune connaissance experte humaine, nous avons utilisé l'apprentissage par renforcement via auto-jeu, où le réseau joue continuellement contre lui-même pour générer ses propres données d'entraînement.

Pour finir, nous avons ajouté une table de Zobrist lors de la phase de jeu pour accélérer les évaluations des positions déjà rencontrées.

2.4 Athénan

En plus de l'implémentation d'AlphaZero, nous avons également implémenté un autre agent basé sur un réseau de neurones, mais qui, cette fois-ci, utilise Minimax pour la sélection d'action, Athénan (Cohen-Solal and Cazenave, 2025).

Pour commencer, le réseau d'Athénan n'a plus besoin de tête de politique, il ne possède donc qu'une tête de valeur pour évaluer les positions. De plus, un changement d'architecture a été fait pour opter pour un réseau R2, qui est détaillé dans le tableau 1.

TABLE 1 – Architecture du réseau de neurones *R2-network* implémenté.

Couche	**Type**	**Détails et paramètres**
1	Convolution	Filtres : 132, Noyau : 3×3, avec padding.
2 – 9	8 Blocs Résiduels	Par bloc : ReLU $\rightarrow$ Conv (3×3, 132 filtres) $\rightarrow$ ReLU $\rightarrow$ Conv (3×3, 132 filtres).
10	Dense + ReLU	518 neurones.
11	Dense + ReLU	518 neurones.
12	Dense (Sortie)	1 neurone (évaluation de la valeur de l'état).

Les différents paramètres de ce réseau ont été choisis afin de satisfaire un choix fait dans l'article d'Athénan, à savoir que le réseau doit contenir environ 5 millions de paramètres.

Ensuite, nous utilisons deux algorithmes de recherche différents pour la phase de jeu et la phase d'entraînement. Durant la phase de jeu, nous utilisons un Unbounded Best-First Minimax (UBFM) qui étend, de manière itérative, l'arbre de recherche en profondeur pour trouver la meilleure action à jouer, le tout utilisant une table de Zobrist pour accélérer les évaluations des positions déjà rencontrées. À la fin du temps imparti pour la recherche, l'action choisie est celle qui a le plus de visites. Durant la phase d'entraînement, nous utilisons Descent Minimax, qui effectue une recherche en profondeur de l'arbre de jeu jusqu'à atteindre un état terminal du jeu. Une fois cette recherche effectuée, l'algorithme utilise un processus appelé Tree Learning pour mettre à jour les poids du réseau de neurones. Contrairement à AlphaZero qui se limite au Terminal Learning (c'est-à-dire l'apprentissage exclusif sur les états constituant la séquence finale de la partie, avec pour cible le résultat final du

match), Athénan apprend sur l'intégralité des états de l'arbre de recherche partiel généré par l'algorithme. De plus, la cible d'apprentissage n'est pas l'issue finale de la partie, mais la valeur Minimax partielle calculée pour chaque état de l'arbre.

Cette approche permet de n'avoir aucune perte d'information issue de la recherche et de générer environ 296 fois plus de données d'entraînement pour un même nombre de parties jouées, permettant ainsi un entraînement plus rapide et plus efficace du réseau de neurones.

3 Résultats et discussions

3.1 Tournoi suisse

Afin d'évaluer les performances de nos différents agents, nous avons organisé un tournoi suisse à 4 rondes. Nous avons inclus dans ce tournoi les agents suivants : Minimax, MCTS, AlphaZero, Athénan, ainsi le Random agent. A chaque ronde, les agents s'affrontent en 1 contre 1 sur 20 parties (10 parties de chaques côtés). Une victoire apporte 3 point, une défaite 0 point, et un match nul 1 point à chaque agent. Etant donné que nous avons 5 agents, un agent ne joue pas à chaque ronde, et reçoit donc un point de bye (ici correspondant à avoir fait un match nul à chaques parties, soit 20 points). Les résultats de ce tournoi sont présentés dans le tableau 2.

TABLE 2 – Résultats détaillés du tournoi suisse (4 rondes, 20 parties par match).

Rang	**Agent**	**R1**	**R2**	**R3**	**R4**	**Total**
1	Minimax	60	36	30	60	**186**
2	Athénan	39	24	20*	57	**140**
3	MCTS	21	60	30	20*	**131**
4	AlphaZero	0	20*	51	3	**74**
5	Random	20*	0	9	0	**29**

* Indique un point de *bye* (20 points accordés d'office).

Pour infomation, l'agent Athénan et AlphaZero ont été entraînés pendant environ 48 heures chacun. Athénan a été entraîné sur CPU (AMD Ryzen 7 5700X avec 64Go de RAM disponnible), tandis qu'AlphaZero a été entraîné sur GPU (NVIDIA GeForce RTX 4060 Ti avec 8Go de VRAM disponnible).

Avec les résultats du tournoi, nous pouvons observer que les approches Minimax, incluant Minimax et Athénan, sont plus performantes que les approches basées sur MCTS, à savoir MCTS et AlphaZero.

De plus, il est intéressant de noter que les agents basés sur des réseaux de neurones, à savoir AlphaZero et Athénan, ont des performances inférieur à celles de leurs homologues basés sur des heuristiques faites à la main, Minimax et MCTS respectivement. Cependant, il est important de souligner que les agents basés sur des réseaux de neurones ont été entraînés pendant une période relativement courte (environ 48 heures), et qu'avec plus de temps d'entraînement, leurs performances pourraient potentiellement s'améliorer. De plus l'entaînement d'AlphaZero a été réalisé avec un équipement beaucoup plus limité que celui utilisé réellement par DeepMind pour entraîner leur agent, ce qui a pu limiter les performances de notre implémentation d'AlphaZero.

4 Conclusion

Références

Quentin Cohen-Solal and Tristan Cazenave. Minimax strikes back, 2025. URL `https://arxiv.org/abs/2012.10700`.

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of go with deep neural networks and tree search. *Nature*, 529 (7587) :484–489, 2016. doi : 10.1038/nature16961. URL `https://doi.org/10.1038/nature16961`.

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm, 2017. URL `https://arxiv.org/abs/1712.01815`.